



GOA ELECTRONICS LIMITED

A Subsidiary of EDC Ltd. (A Government of Goa Undertaking)

Important: The information contained in this document is for exclusive use of Goa Electronics Limited. The information contained in this document should not be passed on to any third party in part or full.

Revision History

Revision No./ Version No.	Date	Prepared by	Approved by	Details of Change
1.0	20-June-2025	Viraj Agni (Software developer) Dattaprasad Ekavade (Software developer)	Anant Yende (CTO)	

Table of Contents

Sr No.	Title	Page No.
1	Introduction	3
2	References on General Coding Standards	3
3	Code Structure & Formatting	3
4	Naming Conventions	4
5	Error Handling	4
6	Styling in React	4
7	Component Structure	5
8	State Management	5
9	Security Best Practices	6
10	API & Web Service Integration	6
11	Testing	6
12	Performance and Maintenance Guidelines	7
13	Type Safety with TypeScript	7
14	Best Practices	7
15	Directory Structure	8
16	Reference Guide on React JS	9

1. Introduction

React is a widely-used JavaScript library for building user interfaces, known for its component-based architecture and efficient rendering system. For new developers, adhering to best practices and standards is essential to create scalable, maintainable, and performant applications. This document defines coding standards specific to React applications while aligning with the "General Software Coding Standards & Best Practices". For general coding principles, refer to the respective sections of the General Coding Standards.

2. References on General Coding Standards

Link: <https://upaygoa.com/geltm/helpndoc/GeneralcodingStandardsBestPracti.html>

1. Code Structure and Formatting (Page 4)
2. Naming Conventions (Page 4)
3. Code Documentation (Page 5)
4. Error Handling (Page 5)
5. Code Re-usability and Modularity (Page 6)
6. Testing (Page 7)
7. Security (Page 7)
8. Code Review (Page 8)
9. Documentation and Commenting (Page 8)
10. Third party libraries / References (Page 8)
11. Web Server & API Standards (Page 9)
12. Best Practices (Page 10)
13. ANNEXURE 'A' - Code Review Checklist (Page 12)

3. Code Structure & Formatting

Refer to Section 2 of the General Coding Standards for Code Structure and Formatting

React-Specific Formatting Guidelines:

- a. Use JSX for rendering UI
- b. Wrap JSX tags in parentheses when returning multiline JSX.
- c. Prefer fragment (`<>...</>`) instead of `<div>` when no additional wrapper is needed.
- d. Use linters (like ESLint) and formatters (like Prettier) to maintain consistent code style.
- e. Group related files, components and hooks logically. Avoid deep nesting.
- f. Use self-closing tags when elements have no children (e.g. `<input />` instead of `<input></input>`).

Example

```
function UserProfile() {  
  return (  
    <>  
    <h1>User Profile</h1>  
    <input type="text" placeholder="Enter name" />  
    </>  
  );  
}
```

4. Naming Conventions

Refer to Section 2 of the General Coding Standards for Naming Conventions

React-Specific Naming Guidelines:

- Component file names should be in PascalCase (UserProfile.js)
- Hooks should start with use (useFetchData.js).
- State variables should be named descriptively (const [userList, setUserList] = useState([])).

Example

```
function useFetchData() {  
  const [data, setData] = useState([]);  
  return data;  
}
```

5. Error Handling

- Use error boundaries for catching component errors (componentDidCatch in class components or ErrorBoundary component for functional components).
- Log errors to a centralized error tracking service.
- Provide meaningful error messages to users.

6. Styling in React

- Use CSS Modules (component.module.css) for scoped styles.
- Use Styled Components for dynamic styling where necessary.
- Maintain consistency in class naming conventions (camelCase for CSS module classes, kebab-case for global styles).
- Avoid inline styles where possible.
- Consider TailwindCSS for rapid and consistent styling with a utility-first approach.

7. Component Structure

- Use functional components over class components.
- Components should be small and reusable.
- Use hooks instead of lifecycle methods.
- Separate UI components (dumb components) from logic-containing components (smart components).
- Maintain a feature-based folder structure.
- Avoid prop drilling, use Context API or state management libraries like Redux.

Advanced Example - Custom Hook:

```
import { useState, useEffect } from 'react';
function useFetchData(url) {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    async function fetchData() {
      const response = await fetch(url);
      const result = await response.json();
      setData(result);
      setLoading(false);
    }
    fetchData();
  }, [url]);
  return { data, loading };
}
```

8. State Management

- Use useState for local component state.
- Use useContext or Redux for global state management.
- Use useReducer when managing complex state transitions.
- Avoid unnecessary re-renders by using useMemo and useCallback where appropriate.
- Optimize state updates to prevent unnecessary re-renders.

9. Security Best Practices

Refer to Section 7 of the General Coding Standards for security guidelines.

React-Specific Security Guidelines:

- Never use dangerouslySetInnerHTML unless absolutely necessary.
- Always sanitize user inputs before rendering.
- Store sensitive data in environment variables (.env).
- Use HTTP-only, secure cookies for authentication.
- Implement CSRF protection for API interactions.

10. API & Web Service Integration

Refer to Section 11 of the General Coding Standards for Web Server & API Standards.

React-Specific API Guidelines:

- Use fetch or axios for API calls.
- Separate API logic into a services/api.js file.
- Use async/await syntax for better readability.
- Handle errors globally using a centralized error handler.
- Implement loading states while fetching data.
- Store sensitive information like API keys securely using environment variables.

Example

```
async function fetchData() {  
  const response = await fetch('/api/data');  
  const result = await response.json();  
  console.log(result);  
}
```

11. Testing

Refer to Section 6 of the General Coding Standards for general testing practices.

React-Specific Testing Guidelines:

- Use Jest and React Testing Library for unit testing.
- Test components in isolation.
- Mock API calls using jest.mock.
- Ensure event handlers and state changes are properly tested.

12. Performance and Maintenance Guidelines

React-Specific Performance and Maintenance Guidelines:

- a. Monitor performance using tools like React Profiler, Lighthouse, and Web Vitals.
- b. Schedule regular maintenance for dependency updates and security patches.
- c. Handle deprecations by tracking React changelogs and updating components proactively.
- d. Maintain a changelog and use react semantic versioning for releases.

13. Type Safety with TypeScript

- a. Define types/interfaces for props, state, function arguments and complex data structures to minimize runtime errors to prevent runtime errors.
- b. Use TypeScript's advanced features to create maintainable and scalable codebases.
- c. Leverage TypeScript's inference capabilities to reduce boilerplate code.
- d. Utilize TypeScript's utility types (Partial, Pick, Record) for cleaner code.
- e. Prefer strict typing (e.g., never for exhaustive checks) to avoid unexpected behavior.

14. Best Practices

Refer to Section 12 of the General Coding Standards for general best practices.

React-Specific Best Practices:

- a. Keep components small and focused.
- b. Avoid using useEffect for unnecessary side effects.
- c. Optimize performance with lazy loading and React.memo.
- d. Always clean up side effects (e.g., remove event listeners, clear timers).
- e. Use React.memo to prevent unnecessary re-renders for pure components.
- f. Use Functional Components and Hooks.
- g. Avoid Anonymous Functions in JSX.
- h. Choose Appropriate State Management.
- i. Keep State Minimal.
- j. Prop Validation.
- k. Use cookies to store authentication tokens or session IDs, ensuring they are marked HttpOnly and Secure.
- l. Store user preferences, caching data, and temporary state in Local Storage for persistence.
- m. Always sanitize inputs, use secure connections (HTTPS), and avoid storing sensitive data in Local Storage. Consider using encryption if you must store sensitive information in Local Storage.



15. Directory Structure

Follow a feature-based structure for organizing files:

```
project-root
├── public                # Public assets (index.html, favicons)
├── src                  # Main source code
│   ├── components      # Reusable UI components
│   │   ├── common      # Shared reusable components (Spinner, Toast, Button)
│   │   ├── DocUpload    # Document upload components (UploadModal, ViewModal)
│   │   └── Revenue      # Revenue module components (Residence, TaxDetails)
│   ├── layout          # Layout-related components (LoaderProvider, Menu)
│   ├── pages           # Route-specific pages
│   │   ├── 404.js       # Custom 404 error page
│   │   └── AccessDenied.js # Page for access denied errors
│   ├── api             # API handlers for different modules
│   │   ├── Revenue      # API services for revenue
│   │   └── DMG          # API services for DMG module
│   ├── styles          # CSS Modules & global styles (Home.module.css)
│   ├── utils           # Utility functions
│   │   ├── callApi.js   # API call handler
│   │   ├── constants.js # Constants used across the app
│   │   ├── encryption.js # Encryption and decryption methods
│   │   └── validations.js # Form and data validation functions
│   ├── App.js          # Main application file
│   ├── index.js        # React DOM entry
│   ├── theme.js        # Theme configuration
│   └── config.js        # App-level configurations
├── .env                # Environment variables
├── .gitignore          # Ignore file for Git
├── package.json         # Project dependencies & scripts
└── README.md           # Project documentation
```

16. Reference Guide on React JS

16.1 Architecture of the React Application

- Root component is build using one or more component.
- Each component can be nested with other component to any level.
- Composition is one of the core concepts of React library. So, each component is build by composing smaller components instead of inheriting one component from another component.
- Most of the components are user interface components.
- React app can include third party component for specific purpose such as routing, animation, state management, etc.
- Different types of Components
 - a) Functional Components: React components that are written as functions. They are simple, lightweight, and can use hooks for state and side-effects.
 - b) Class Components: Older style of writing React components. Though less common now with the introduction of hooks, class components still have their place in legacy code.
 - c) UI Components: These handle the presentation logic and are usually stateless (e.g., buttons, forms, modals).
 - d) Container Components: Also called "Smart Components", these contain the state logic and often handle data fetching and passing props down to UI components.
- JSX (JavaScript XML)
 - a) React uses JSX, a syntax extension for JavaScript, that allows writing HTML elements in JavaScript. It helps in creating React elements and components.
- State Management
 - a) React manages state within components using the useState hook or the this.state in class components.
 - b) For larger applications, state management libraries like Redux, Context API, or MobX are used to share state across components.
- Hooks
 - a) useState: For managing state.
 - b) useEffect: For handling side-effects (data fetching, subscriptions, etc.).
 - c) useContext: To share state without passing props explicitly (often used with Context API).
 - d) useReducer: Alternative to useState for more complex state logic (similar to Redux).

16.2 Functions & Callbacks in React Application

- Use Function Declarations or Arrow Functions consistently, especially for defining event handlers and callbacks.
- Functional components should be written using arrow functions or function expressions for readability.

Example:

Correct (Arrow function):

```
const handleClick = () => {
  console.log('Button clicked');
};
```

Correct (Arrow function):

```
const handleClick = () => {
  console.log('Button clicked');
};
```

- Event Handlers and Callbacks
 - a. Event handlers: Functions used to handle user inputs like clicks, form submissions, etc.
 - b. Callback functions: Functions passed as arguments to other functions to be executed later, typically after some event or async operation.
 - c. Bind event handlers properly when working with class-based components (e.g., using `.bind()` in constructors).
- Managing State with Callbacks
 - a. When using state in React, ensure callbacks are properly passed to update or retrieve state.
 - b. Functional components with state (using hooks): Callbacks that modify state can be declared using the `useState` hook. Ensure you avoid mutating the state directly and always use the setter function.
- Avoid Inline Functions in JSX
 - a. Avoid declaring functions inline in JSX. While it is common to see anonymous functions within JSX tags (like event handlers), it's better for performance and readability to define them outside the JSX.

Example:

```
const handleClick = () => {
  console.log('Button clicked');
};

return <button onClick={handleClick}>Click me</button>;
```

- Handling Multiple Parameters in Callbacks
 - a. For callbacks that require multiple parameters, ensure the function signature is clean and concise. Use destructuring when the arguments are objects or arrays.
- Use of `useCallback` for Performance Optimization
 - a. If a function is being passed as a prop to child components, it can be memoized using the `useCallback` hook to avoid unnecessary re-renders, especially when the function depends on state or props.
- Avoiding Nested Functions or Callbacks

- a. Avoid deeply nested callbacks in JSX or within other functions, as it makes code harder to understand and debug.
 - b. Keep functions as flat as possible, and if necessary, use helper functions to break them into smaller parts.
- Returning Early in Functions
 - a. Use early returns for functions that require conditional logic, especially in React components. It helps to minimize nesting and makes the code easier to follow.
- Callback Functions in Event Handlers
 - a. When using event handlers like `onClick`, `onChange`, etc., ensure callback functions are properly passed and the right parameters are handled.

16.3 Loops in React Application

- Use `.map()` for Rendering Lists
 - a. Use `.map()` to iterate over arrays and render JSX.
 - b. Always provide a unique key for each element to avoid rendering issues.
 - c. Avoid using `.map()` for complex logic or side effects.
 - d. Example: `{items.map((item, index) => <li key={index}>{item}</li>)}`
- Use for Loops for Complex Logic
 - a. Use for loops for data manipulation or logic, not for rendering JSX directly.
 - b. If you use for to generate JSX, ensure the result is returned as an array. Use for Loops for Complex Logic
- Conditional Rendering with Loops
 - a. Use conditional rendering (ternary operators or if statements) for different UI outputs..
 - b. Avoid rendering loops in conditions that result in an empty array or component.
 - c. Example: `{items.length > 0 ? items.map(item => <li key={item.id}>{item.name}</li>) : <li>No items available</li>}`

16.4 Encryption in React Application

- Install the `crypto-js` Library for AES Encryption: Install `crypto-js` using the command `npm install crypto-js`. This library will help you perform AES encryption and other cryptographic operations.
- Refer `General_Software_Coding_Standards` (8.2 DataEncryption/Protection) for more information.

16.5 File Upload in React Application

- File Selection: Use a standard HTML `<input type="file" />` or a custom React component for selecting files. The input should restrict file selection based on file types (e.g., images, documents) using the `accept` attribute.
- Client-Side Validation: File Type Validation: Ensure only allowed file types are uploaded (e.g., images, PDFs). This can be done by checking the MIME type or file extension.

- **File Size Validation:** Check the file size before upload. Reject files that exceed the allowed size limit (e.g., 2 MB).
- **File Integrity Check:** For certain files, you can validate their structure (e.g., image file header validation) to ensure they are not corrupted or malformed.
- **File Preview:** For images or media files, generate a preview on the client-side before uploading, allowing the user to confirm the file before submitting.
- **Security Against Malicious Content: Client-Side Filtering:** Prevent files with dangerous extensions (e.g., .exe, .bat, .js) from being selected and uploaded.
- **Use Blob URLs for Security:** Avoid using the src attribute with file paths directly. Use `URL.createObjectURL()` for generating a temporary URL for file previews to prevent exposing paths to potential threats.
- **Limit File Upload Types:** Always restrict the type of file a user can upload by specifying a list of acceptable MIME types or file extensions (e.g., .jpg, .png, .pdf).

16.6 Login Authentication & Session in React Application

- **Salt Generation for Password Encryption**
 - a) Salt should be combined with the password before encryption.
 - b) Ensure the salt is securely generated and stored.
- **Login Authentication**
 - a) Use the encrypted password and salt combination for authentication in the Login API.
 - b) On successful login, remove the temporary token and store the user's details securely in localStorage.
- **All session objects to be disposed on logout or time-out function** Avoid using session object for the local data in the form.
- **Use Session Storage Only When Necessary:**
 - a) Store temporary data (e.g., non-authenticated states) in session-storage.
 - b) Long-term data (e.g., preferences) can go in cookies or local-storage.
- **Session Timeout**
 - a) **Track User Activity:** Detect mouse movements, keyboard presses, and other user interactions to reset the session timer. Use JavaScript to listen for mousemove, keydown, scroll, and click events.
 - b) **Set Session Timeout:** After a specified period of inactivity (e.g., 15 minutes), the session will expire, and the user will be logged out automatically.
 - c) **Implementation Steps:** Set up an interval function that checks if the user has been inactive for longer than the `timeoutDuration`.
 - d) **Clear User Session:** When the session expires, remove any authentication tokens from localStorage, sessionStorage, or any global state (e.g., Redux/Context).
 - e) **Redirect to Login:** After session expiration, redirect the user to the login page or trigger a logout process.

16.7 Cookies In React Application

- To handle cookies easily in a React application, we can use the js-cookie library.
- Install js-cookie: `npm install js-cookie`
- You can store cookies using `Cookies.set()`. You can specify options like `expires` to define how long the cookie should last.
- To get the value of a cookie, use `Cookies.get()`.
- To remove a cookie, you can use `Cookies.remove()`.
- Use Case for Cookies in React Local Storage:
 - a) Authentication: Store JWT tokens to keep a user logged in across sessions.
 - b) Session Management: Track user login status and manage authentication.
 - c) User Preferences: Store settings like theme or language preferences.
- Security Considerations for Cookies:
 - a) Use Secure Flag: Ensure cookies are only sent over HTTPS.
 - b) Use HttpOnly Flag: Prevent JavaScript access to sensitive cookies like authentication tokens.
 - c) Use SameSite Flag: This protects against Cross-Site Request Forgery (CSRF) attacks by ensuring cookies are only sent with requests originating from the same site.

16.8 Local Storage In React Application

- You can store data in Local Storage using the `localStorage.setItem()` method.
- Data is stored as key-value pairs.
- To retrieve data, use `localStorage.getItem()`
- To remove an item, use `localStorage.removeItem()`
- Use Case for Local Storage in React:
 - a) User Preferences: Store non-sensitive data like theme, language preferences, or layout settings.
 - b) Offline Caching: Cache data from API requests to improve performance and allow the app to work offline.
 - c) State Persistence: Store temporary application state between page reloads, such as form data.
- Security Considerations for Local Storage:
 - a) Avoid Storing Sensitive Information: Local Storage is accessible via JavaScript, so avoid storing sensitive data like authentication tokens or passwords.
 - b) Vulnerable to XSS: Be cautious of Cross-Site Scripting (XSS) attacks where attackers can inject malicious scripts to access Local Storage.

16.9 Validations in React Application

- Date to be validated with server date for comparisons
- Date format to be ensured to prevent any data loss during conversion.
- The search or input pages on public domain should have a captcha code implementation.
- Validation summary to be compulsorily displayed in pop-up and in form footer
- Confirm alert to be shown for following critical actions like Save ,Update ,Delete, Cancel, Back, etc

- Back or cancel button should intimate user about loss of edited information if proceeded with cancel/back
- After action button is clicked the action button should be disabled or hidden to prevent user from submitting the same action multiple times
- After the response is received from the server the same need to be presented to the user before navigating to a new page unless otherwise specified by client for auto redirection
- Proper success or error panel to be shown with specific message to alert user about the processing outcome
- Every control should have the field length as per the data structure definition
- Control height and width to be as per the type of the data being captured or as per the UI/UX theme of the project and same uniformity to be ensured across complete Project
- Avoid using unnecessary post backs.